

EV Adoption & Charging Forecasting System Deployment Plan – Flask Web Application

This document outlines the **step-by-step deployment plan** for my EV forecasting system. The plan covers both statewide and county-level Flask applications and the exact sequence of tasks required for a successful deployment.

Prepared by: Judy Cheng
Capstone Project – EV Forecasting & Analytics

Note: This project uses a **free Flask deployment** (Render.com free tier).

1. Introduction

This deployment plan provides the sequenced steps for deploying an interactive Flask application that supports:

1. Statewide forecasting using best-fit cubic regression.
2. County-level forecasting using 39 localized models.

2. Deployment Objectives

Key Objectives:

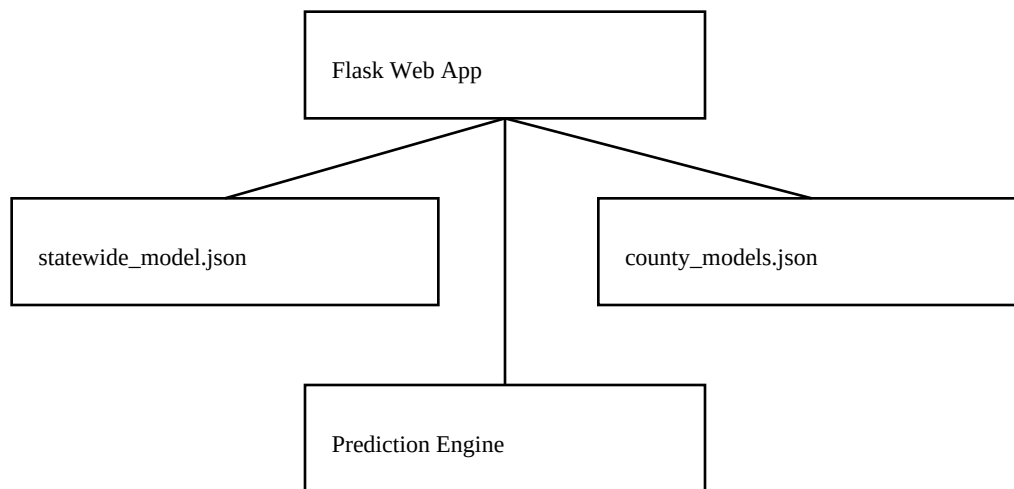
- Deliver a real-time EV forecasting web interface.
- Enable predictive outputs from charger counts.
- Provide both statewide and county-level modes.
- Use a free Flask hosting platform (Render.com).

Why Flask?

- Light, fast, and Python-native
- Easy routing and API creation
- Ideal for small to medium machine learning apps

3. System Architecture Overview

This diagram shows how Flask, the prediction engine, and the JSON model files interact.



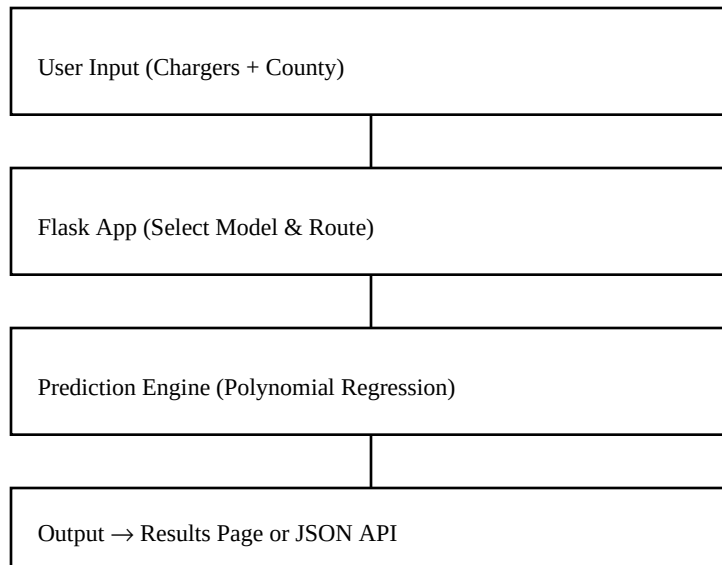
4. Statewide Flask Deployment – Step-by-Step Plan

1. Extract statewide regression coefficients.
2. Save coefficients into statewide_model.json.
3. Build Flask route: /predict/statewide.
4. Create input form for "total chargers".
5. Connect prediction engine to compute EV & adoption.
6. Display forecast on a results page.
7. Add /api/statewide JSON endpoint.
8. Deploy the statewide portion on Render.com free tier.

5. County-Level Flask Deployment – Step-by-Step Plan

1. Calculate 39 sets of county-level regression coefficients.
2. Save them into county_models.json.
3. Build Flask route: /predict/county.
4. Add dropdown for selecting a county.
5. Add numeric input for county charger projection.
6. Load county-specific coefficients dynamically.
7. Display county-level forecast results.
8. Add /api/county JSON endpoint.
9. Deploy together with statewide app on Render.com.

Forecast Workflow Diagram



6. Deployment Options

- A. Local deployment for development
- B. Render.com free tier for final deployment
- C. AWS EC2 as an advanced option
- D. Docker packaging (optional)

7. Deliverables

- Flask app (statewide + county)
- JSON model files (statewide_model.json, county_models.json)
- README documentation
- Screenshots of UI
- Public free deployment URL (Render.com)